

Continuous Assessment II

Paul Haskins, 22338905

April 8, 2025

Question 1

0.0.1 (a)

Here we have no interaction between the particles (no potential) so the internal energy of the system is the total kinetic energy.

$$U = \frac{m}{2} \sum_{i=1}^N v_i^2 + \frac{MV^2}{2} \quad (1)$$

0.0.2 (b)

We have the standard equation for entropy for a classical system in 3D given by

$$H = U + PV \quad (2)$$

As we are working in 1D we have to identify the analogous quantities namely.

Volume $\rightarrow$ Length and Pressure $\rightarrow$ Force

Here we can identify the only force F we are concerned with is that of the piston and we have a length X . Subbing in our previous expression for the internal energy we have

$$H = \frac{1}{2} \left[m \sum_{i=1}^N v_i^2 + MV^2 \right] + FX \quad (3)$$

0.0.3 (c)

Before a particle hits a boundary there is no forces acting on it thus it has constant velocity. Once it hits a boundary it should bounce back.

If we take the i th particle we have the equation of motion

$$x_i(t) = x_i(0) + \frac{dx_i}{dt} (t - t_0) \quad (4)$$

0.0.4 (d)

We have a force F from the piston coming from the right with the piston moving at $V(+\hat{x})$ with the force once the piston to be $-F(\hat{x})$.

Thus we get the EOM.

$$F = ma \implies -F = M \frac{d^2 X}{dt^2} \quad (5)$$

$$\frac{dX}{dt} = - \int_{t_0}^t \frac{F}{M} dt \quad (6)$$

$$\frac{dX}{dt} = -\frac{F}{M} (t - t_0) + \left(\frac{dX}{dt} \right) \Big|_{t_0} \quad (7)$$

$$X(t) = \int_{t_0}^t \left(-\frac{F}{M} (t - t_0) + \left(\frac{dX}{dt} \right) \Big|_{t_0} \right) dt \quad (8)$$

$$u = t - t_0$$

$$X(t) = \int_0^{t-t_0} \left(-\frac{F}{M} (u) + \left(\frac{dX}{dt} \right) \Big|_{t_0} \right) du \quad (9)$$

So we arrive at our equation of motion

$$X(t) = -\frac{F}{2m} (t - t_0)^2 + V^0 (t - t_0) + X^0 \quad (10)$$

0.0.5 (e)

The time of the collision is t so the waiting time is defined as $\tau = t - t_0$. If we approximate the piston and the particle as small then they will 'colide' when their positions are equal.

$$X(t) = x(t) \quad (11)$$

We can use the equations we obtained

$$x + v\tau = -\frac{F}{2M}\tau^2 + V\tau + X \quad (12)$$

$$-\frac{F}{2M}\tau^2 + (V - v)\tau + X - x = 0 \quad (13)$$

Using the standard quadratic formula

$$\tau = \frac{-(V - v) \pm \sqrt{(V - v)^2 - 4\left(-\frac{F}{2M}\right)(X - x)}}{2\left(-\frac{F}{2M}\right)} \quad (14)$$

Simplifying we get

$$\tau = \frac{M}{F} \left[V - v \pm \sqrt{(V - v)^2 - 2\frac{F}{M}(X - x)} \right] \quad (15)$$

Here we have two solutions which arise from the quadratic nature of our formula.

We need to consider the cases. If $v \geq V$ then in order for τ to be positive we must take the positive branch.

On the other hand if $v < V$ the positive branch keeps τ positive.

The negative branch can be investigated by letting $\tau = 0$. We can relabel $V - v = a$, $b = \frac{2F}{M}(x - X)$. Thus we have

$$0 = a - \sqrt{a^2 - b} \quad (16)$$

$$a = \sqrt{a^2 - b} \quad (17)$$

Thus b must equal zero, $\implies$ either $M = \infty$ or $X - x = 0$, the former is impossible and the latter is never actually zero as they are interacting. Thus the negative branch is unphysical and we can drop this branch. This gives us our final result.

$$\tau = \frac{M}{F} \left[V - v + \sqrt{(V - v)^2 - 2\frac{F}{M}(X - x)} \right] \quad (18)$$

Now we consider when the particle is moving to the left. It would first have an elastic collision with the wall the come back. This distance traveled $2x$ to wall and back is the same as if we were to reflect the particle about the wall the remove the wall and let it move in the same direction. As we are reflecting space here we have the following coordinate transformation $X \rightarrow -X$ which leads to $V \rightarrow -V$ and $F \rightarrow -F$.

Thus we are essentially solving the same problem but with this transformations implemented. We can get a new equation for τ from the quadratic formula to be

$$\tau = \frac{(V + v) \pm \sqrt{(-V - v)^2 + 4\left(\frac{F}{2M}\right)(X + x)}}{2\left(\frac{F}{2M}\right)} \quad (19)$$

$$\tau = \frac{M}{F} \left[V + v + \sqrt{(V + v)^2 + 2\frac{F}{M}(X + x)} \right] \quad (20)$$

0.0.6 (f)

For a right moving particle.

We can use conservation of momentum

$$MV + mv = MW + mw \quad (21)$$

We can re-arrange for the Final velocity of the piston

$$W = V + \frac{m(v-w)}{M} \quad (22)$$

Now we can also consider that fact we have elastic collisions and thus can use the conservation of energy

$$\frac{1}{2}(mv^2 + MV^2) = \frac{1}{2}(mw^2 + MW^2) \quad (23)$$

If we plug in our expression for W and cancel the half

$$mv^2 + MV^2 = mw^2 + M \left[V + \frac{m(v-w)}{M} \right]^2 \quad (24)$$

$$mv^2 + MV^2 = mw^2 + M \left[V^2 + \frac{2mV(v-w)}{M} + \frac{m^2(v^2 - 2vw + w^2)}{M^2} \right] \quad (25)$$

We can expand and multiply both sides by $\frac{M}{m}$ and bring everything to the right.

$$0 = (M+m)w^2 - 2(MV + mv)w + (mv^2 + 2MvV - Mv^2) \quad (26)$$

We can identify this is of the form of a quadratic in w . Which gives us

$$w = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a} \quad (27)$$

where we can identify $a = M + m$, $b = -2(MV + mv)$ and $c = (mv^2 + 2MvV - Mv^2)$. We have the discriminant $\Delta = b^2 - 4ac$

$$\Delta = 4(MV + mv)^2 - 4(m + M)(mv^2 + 2MvV - Mv^2) \quad (28)$$

$$\Delta = 4M^2V^2 + 8MVmv + 4m^2v^2 - 4m^2v^2 - 8mMvV + 4mMv^2 - 4Mmv^2 - 8M^2vV + 4M^2v^2 \quad (29)$$

$$\Delta = 4M^2V^2 - 8M^2vV + 4M^2v^2 \quad (30)$$

Thus we can write

$$w = \frac{MV + mv \pm \sqrt{M^2(V^2 - 2vV + v^2)}}{(M + m)} \quad (31)$$

$$w = \frac{MV + mv \pm \sqrt{M^2(V - v)^2}}{M + m} \quad (32)$$

$$w = \frac{MV + mv \pm M(V - v)}{M + m} \quad (33)$$

If we take the positive branch

$$w = \frac{MV + mv + M(V - v)}{M + m} \quad (34)$$

$$w = \frac{2MV + (m - M)v}{m + M} \quad (35)$$

and if we take the negative branch

$$w = \frac{MV + mv - M(V - v)}{M + m} = v \quad (36)$$

The positive branch describes the scenario where the particle collides with the piston and bounces back. The second corresponds to when the particle doesn't interact with the piston. We are interested in the former.

We can sub our expression for w into our expression for W

$$W = V + \frac{m(v-w)}{M} \quad (37)$$

$$W = V + \frac{m}{M} \left(v - \frac{2MV + (m - M)v}{m + M} \right) \quad (38)$$

$$W = V + \frac{m}{M} \left(\frac{v(M+m)}{M+m} - \frac{2MV + (m-M)v}{M+m} \right) \quad (39)$$

$$W = V + \frac{m}{M} \left(\frac{2Mv - 2MV}{m+M} \right) \quad (40)$$

$$W = \frac{2mv - 2mV + mV + MV}{m+M} \quad (41)$$

$$W = \frac{2mv + (m-M)v}{M+m} \quad (42)$$

For the case of a particle moving to the left we can apply the same reflection idea as before with the same transformation (negative signs). This gives us

$$w = \frac{-2MV + (m-M)v}{m+M} \quad (43)$$

and for the piston

$$W = \frac{-2mv + (M-m)V}{m+M} \quad (44)$$

Question 2

(a)

```
z = np.zeros((1000, 2))
```

Figure 1: One line statement that creates a 1000x2 array.

This statement requires for numpy to be imported as I am using 'np.zeros'.

(b)

```
def sum(arr, a, b):
    return np.sum(arr**a), np.sum(arr**b)
```

Figure 2: A function with input of an array and two floating point numbers a and b .

```
randoma = np.random.rand(10)
print(randoma)
x, y = sum(randoma, 2, 3)
```

Figure 3: 2.2.1.png

(c)

```
gdata = np.random.randn(500)

plt.figure()
plt.hist(gdata, bins=20, range=(-3,3)) # Show 3 sigma
plt.title("Histogram w/ 500 samples, 3 sigma range")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.hist(gdata, color='red')
plt.show()
```

Figure 4: 2.3.png

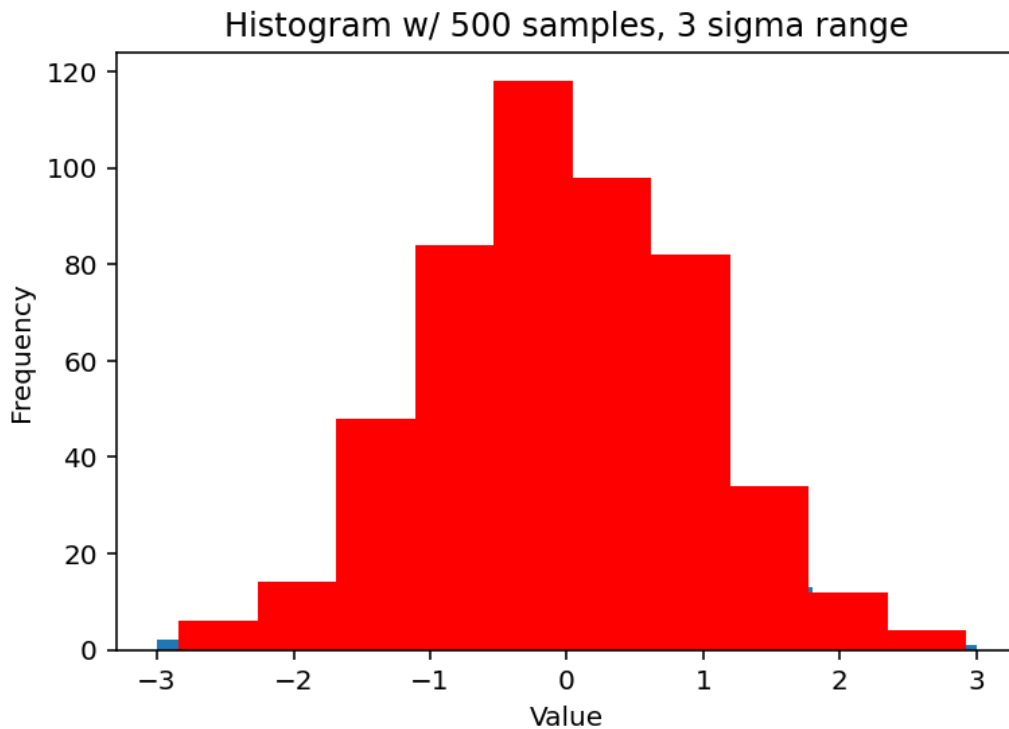


Figure 5: h1.png

(d)

```
for i in range(3):
    rand_choice = np.random.choice([10, -10], size=20)
    print(f"Random array #{i+1}:\n", rand_choice, "\n")
```

Figure 6: 2.4.png

```
Random array #1:
[-10  10 -10 -10 -10 -10 -10 -10 -10 -10  10  10 -10  10  10  10  10 -10
 10 -10]

Random array #2:
[-10 -10  10  10  10 -10 -10 -10  10  10 -10  10  10  10  10 -10 -10  10
-10 -10]

Random array #3:
[ 10 -10  10  10 -10 -10  10  10  10 -10  10 -10  10  10  10 -10  10 -10
-10 -10]
```

Figure 7: 2.4.1.png

(e)

```
z = []

for i in range(50):
    count = 0
    while count < 100:
        num = np.random.uniform(-50, 50)
        z.append(num)
        if count % 25 == 0:
            print(f" {num:.2f}")
        count += 1

plt.figure()
plt.hist(z, bins=30)
plt.title("Histogram w/ 5000 values (uniform distribtion)")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.hist(z, color='red')
plt.show()
```

Figure 8: 2.5.png

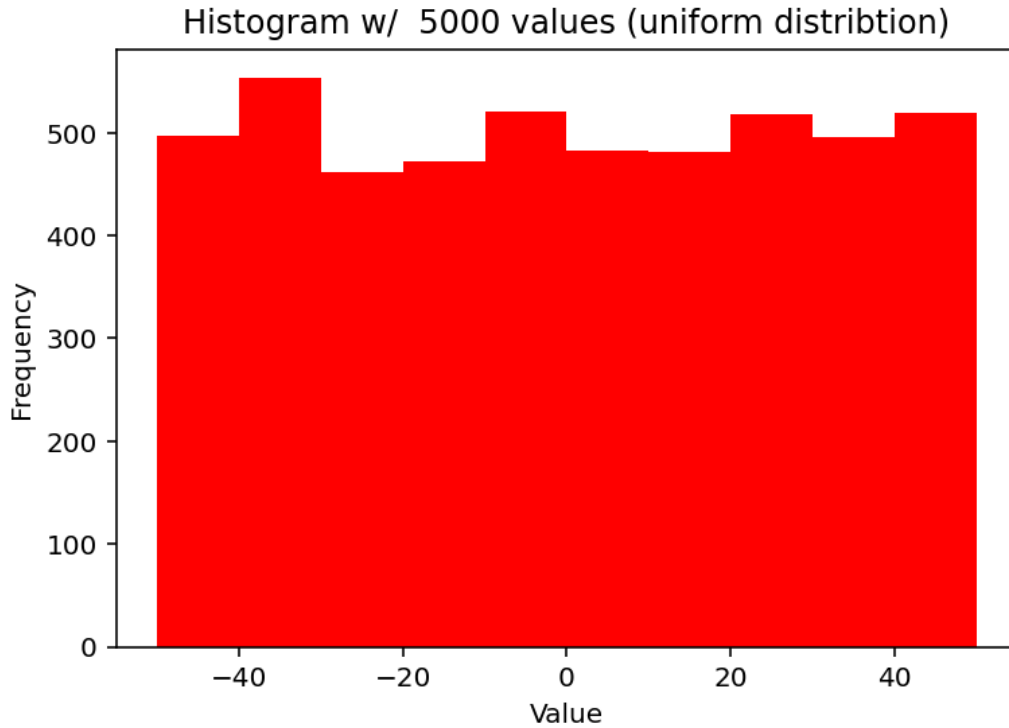


Figure 9: h2.png

Question 3

We have the ideal gas law for 3D

$$PV = NRT = NkT \quad (45)$$

$$(46)$$

We modify it for our 1D case $P \rightarrow F$ and $V \rightarrow X$

$$FX = NkT \quad (47)$$

Given that the initial position is twice the equilibrium position

$$F \frac{X_0}{2} = NkT \quad (48)$$

$$X_0 = \frac{2NkT}{F} \quad (49)$$

$$(50)$$

We are given that the each particle has kinetic energy given by

$$K = \frac{1}{2}mv^2 = \frac{1}{2}kT \quad (51)$$

$$v^2 = \frac{kT}{m} \quad (52)$$

We want to thus find the expectation value of the velocity $\langle v \rangle$

As we have a uniform distribution of particles traveling in $+\hat{x}$ and $-\hat{x}$. The mean velocity $V_\mu = 0$. We have a Gaussian distribution of position x we know from the Gaussian distribution that the EV of x^2 is the variance

which is the square of the standard deviation.

$$Var(x^2) = \sigma^2 = \langle x^2 \rangle \quad (53)$$

$$\Rightarrow \sigma = \sqrt{\frac{kT}{m}} \quad (54)$$

Note throughout the following parts I will display code and graphs which call on the following Utility functions I will display below.

```
def elastic_collision(u, U, m, M):
    """
    Compute post-collision velocities for an elastic collision between a particle
    (velocity u, mass m) and the piston (velocity U, mass M).
    """
    v_new = (m - M)/(m + M)*u + (2*M/(m + M))*U
    U_new = (2*m/(m + M))*u + (M - m)/(m + M)*U
    return v_new, U_new
```

Figure 10: We define a function to calculate the final velocities of a particle and the piston after a given collision.

```
def quadratic_solver(A, B, C):
    """
    Solve quadratic equation A*dt^2 + B*dt + C = 0 for the smallest positive dt.
    Returns dt if found; otherwise returns np.inf.
    """
    if abs(A) < 1e-14:
        # Then it's linear: B*dt + C = 0
        if abs(B) < 1e-14:
            return np.inf
        dt = -C / B
        return dt if dt > 1e-12 else np.inf
    disc = B**2 - 4*A*C
    if disc < 0:
        return np.inf
    sqrt_disc = np.sqrt(disc)
    dt1 = (-B - sqrt_disc)/(2*A)
    dt2 = (-B + sqrt_disc)/(2*A)
    dt_candidates = [dt for dt in (dt1, dt2) if dt > 1e-12]
    return min(dt_candidates) if dt_candidates else np.inf
```

Figure 11: We define a function to solve quadratics by taking the min dt and taking cases of the coefficients. If A or B are small we approximate as linear or a quadratic binomial respectively. We also deal with negative discriminants and then take the minimum solution. This is only an approximation as we can't find the true solution numerically in this manner in a finite time span.

```
def gaussian(x, A, mu, sigma):
    """
    Simple Gaussian function for curve fitting (used in Part 3(c)).
    """
    return A * np.exp(-(x - mu)**2 / (2*sigma**2))
```

Figure 12: Here a Gaussian fit is defined

```
def compute_weighted_average(x, t):
    """
    Compute weighted average of array x with weights given by time step differences dt.
    """
    dt = np.diff(t)
    return np.sum(x[1:] * dt) / np.sum(dt)
```

Figure 13: Here a weighted average function is defined.

Main Function

We initially randomise the positions and give the velocities.

```
def simulate_piston_event_driven(N, m, M, T0, F, t_max,
                                init_vel_dist='maxwell',
                                interactions=False,
                                dt_output=0.01,
                                k_B=1,
                                init_piston_at_eq=False,
                                record_at_collision_events=False):
    """
    Event-driven simulation of the 1D piston-gas system.

    In dimensionless mode, pass dimensionless m, M, T0, F, k_B=1, etc.
    In SI mode, pass real SI values.

    Returns a dict with keys:
    't', 'X', 'piston_v', 'enthalpy', 'snapshots', 'X_eq', 'positions', 'velocities'
    """
    X_eq = N * k_B * T0 / F
    X0 = X_eq if init_piston_at_eq else 2.0 * X_eq
    left_wall = 0.0
    X = X0
    piston_v = 0.0
    a_piston = -F / M

    # Initialize particle positions uniformly in [left_wall, X]
    positions = np.random.uniform(left_wall, X, size=N)

    # Initialize velocities
    if init_vel_dist == 'maxwell':
        sigma = np.sqrt(k_B * T0 / m)
        velocities = np.random.normal(0.0, sigma, size=N)
    elif init_vel_dist == 'bipolar':
        v0 = np.sqrt(k_B * T0 / m)
        velocities = np.concatenate(
            (np.full(N // 2, -v0), np.full(N - N // 2, v0))
        )
        np.random.shuffle(velocities)
    else:
        raise ValueError("Unknown init_vel_dist. Use 'maxwell' or 'bipolar'.")

    # Simulation time + output arrays
    t = 0.0
    t_series = [t]
    X_series = [X]
    V_series = [piston_v]
    U_internal = np.sum(0.5*m*velocities**2) + 0.5*M*(piston_v**2)
    enthalpy_series = [U_internal + F*X]
    snapshots = {}

    if record_at_collision_events:
        # Record state at each collision event
        while t < t_max:
            dt_wall = np.full(N, np.inf)
            mask_wall = (velocities < 0)
            dt_wall[mask_wall] = -positions[mask_wall]/velocities[mask_wall]

            dt_piston = np.full(N, np.inf)
            mask_pist = (velocities > 0)
            for i in np.where(mask_pist)[0]:
```

```

        A = 0.5*a_piston
        B = piston_v - velocities[i]
        C = X - positions[i]
        dt_piston[i] = quadratic_solver(A, B, C)

    dt_event = min(dt_wall.min(), dt_piston.min())
    if dt_event == np.inf:
        break # no further collisions

    # Move forward dt_event
    positions += velocities*dt_event
    X += piston_v*dt_event + 0.5*a_piston*(dt_event**2)
    piston_v += a_piston*dt_event
    t += dt_event
    positions = np.minimum(positions, X)

    # Check which collision is next
    if dt_wall.min() <= dt_piston.min():
        idx_wall = np.argmin(dt_wall)
        velocities[idx_wall] = -velocities[idx_wall]
    else:
        idx_pist = np.argmin(dt_piston)
        u_ = velocities[idx_pist]
        U_ = piston_v
        new_u, new_U = elastic_collision(u_, U_, m, M)
        velocities[idx_pist] = new_u
        piston_v = new_U

    # Record
    t_series.append(t)
    X_series.append(X)
    V_series.append(piston_v)
    U_internal = np.sum(0.5*m*velocities**2) + 0.5*M*piston_v**2
    enthalpy_series.append(U_internal + F*X)

else:
    # Record at fixed dt_output intervals
    next_output_time = dt_output
    snapshot_times = np.linspace(0, t_max, 7)
    snapshot_index = 0

    while t < t_max:
        dt_wall = np.full(N, np.inf)
        mask_wall = (velocities < 0)
        dt_wall[mask_wall] = -positions[mask_wall]/velocities[mask_wall]

        dt_piston = np.full(N, np.inf)
        mask_pist = (velocities > 0)
        for i in np.where(mask_pist)[0]:
            A = 0.5*a_piston
            B = piston_v - velocities[i]
            C = X - positions[i]
            dt_piston[i] = quadratic_solver(A, B, C)

        dt_event = min(dt_wall.min(), dt_piston.min())
        dt_out = next_output_time - t

        if dt_out < dt_event:
            dt = dt_out
            collision = False
        else:

```

```

        dt = dt_event
        collision = True

    positions += velocities*dt
    X += piston_v*dt + 0.5*a_piston*(dt**2)
    piston_v += a_piston*dt
    t += dt
    positions = np.minimum(positions, X)

    if collision:
        if dt_wall.min() <= dt_piston.min():
            idx_wall = np.argmin(dt_wall)
            velocities[idx_wall] = -velocities[idx_wall]
        else:
            idx_pist = np.argmin(dt_piston)
            u_ = velocities[idx_pist]
            U_ = piston_v
            new_u, new_U = elastic_collision(u_, U_, m, M)
            velocities[idx_pist] = new_u
            piston_v = new_U

    # Output
    if (abs(t - next_output_time) < 1e-8) or (t > next_output_time):
        t_series.append(t)
        X_series.append(X)
        V_series.append(piston_v)
        U_internal = np.sum(0.5*m*velocities**2) + 0.5*M*(piston_v**2)
        enthalpy_series.append(U_internal + F*X)
        next_output_time += dt_output

    if snapshot_index < len(snapshot_times) and t >= snapshot_times[
        snapshot_index]:
        snapshots[snapshot_times[snapshot_index]] = velocities.copy()
        snapshot_index += 1

    return {
        't': np.array(t_series),
        'X': np.array(X_series),
        'piston_v': np.array(V_series),
        'enthalpy': np.array(enthalpy_series),
        'snapshots': snapshots,
        'X_eq': X_eq,
        'positions': positions,
        'velocities': velocities
    }

```

(a)

One can find the expected final position by using the conservation of Enthalpy

$$H_i = U_i + F_i X_i = 500 + 10(200) = 2500 = H_f \quad (55)$$

$$H_f = U_f + F X_f = \frac{1}{2} N k \left(\frac{F X_f}{N K} \right) + F X_f = \frac{3}{2} F X_f \quad (56)$$

$$X_f = \frac{2}{3} \frac{H_f}{F} = \frac{2}{3} \frac{2500}{10} = 166.\dot{6} \quad (57)$$

Here we used the equipartition of energy $K = \frac{1}{2} k T$

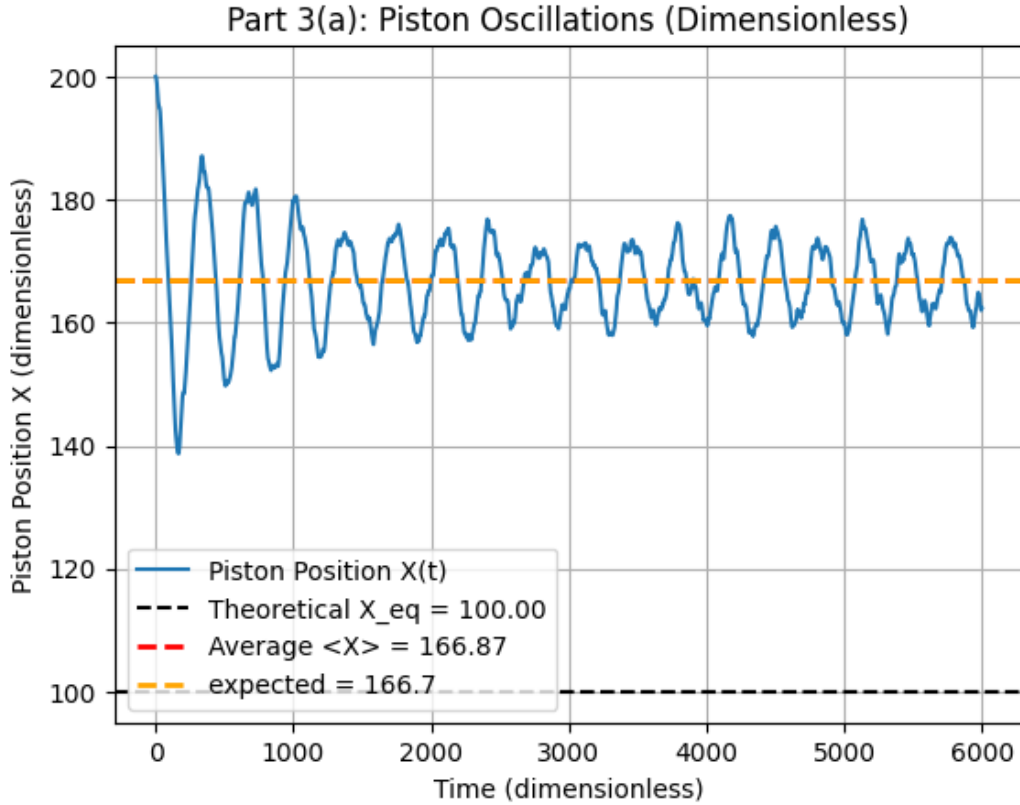


Figure 14: Plot of the Piston position versus time $X(t)$. The average is plotted in red. We define the average to be the equilibrium as it is precisely the point the oscillation is about. We see a disalignment between the prediction of the Ideal gas law and the simulation.

Here we can see as time goes on the amplitude decrease this is due to the exponential dampening that occurs from energy transfer to particles. The pattern resembles a series of cosine waves (starts at max) and clearly oscillates about an equilibrium position of $166.\dot{6}$. It is the very fact that as we are starting from the max that causes the observed equilibrium not to coincide with the prediction by the ideal gas law. This observed equilibrium position disagrees with the theoretical result given by the ideal gas law at $T_0 = 1$ which is $X_{eq} = 100$.

In order to Calculate the expected piston equilibrium position for the Initial conditions of $N = 1000$, $F = 10$, $T_0 = 1$, $k = 1$ We can use the 1D Ideal gas law

$$F X_{eq} = N k T \quad (58)$$

$$X_{eq} = \frac{N k T}{F} \quad (59)$$

$$X_{eq} = \frac{1000 \cdot 1 \cdot 1}{10} = 100 \quad (60)$$

We can see this plotted as a black dashed horizontal line.

(b)

We have plots of the piston's position versus time for $F \in \{0.1, 0.3, 1, 3, 25, 50, 75, 100\}$

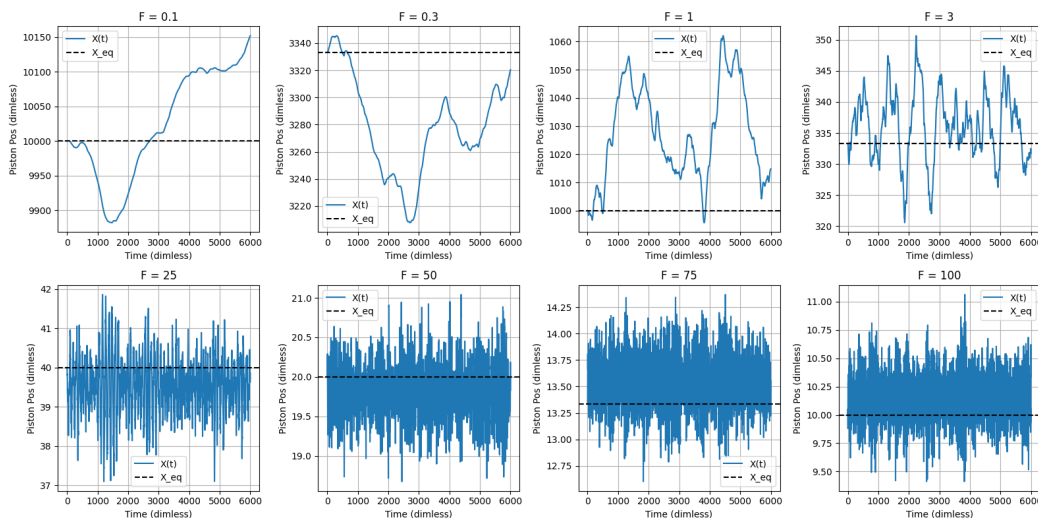


Figure 15: Plot of piston's positions versus time for various forces.

We can see that as we increase the force we see there is more oscillation and thus more of the phase space is realised. This coincides with the expected behavior of increasing the pressure in a 3D system. Clearly we see the amplitude decrease with increasing force.

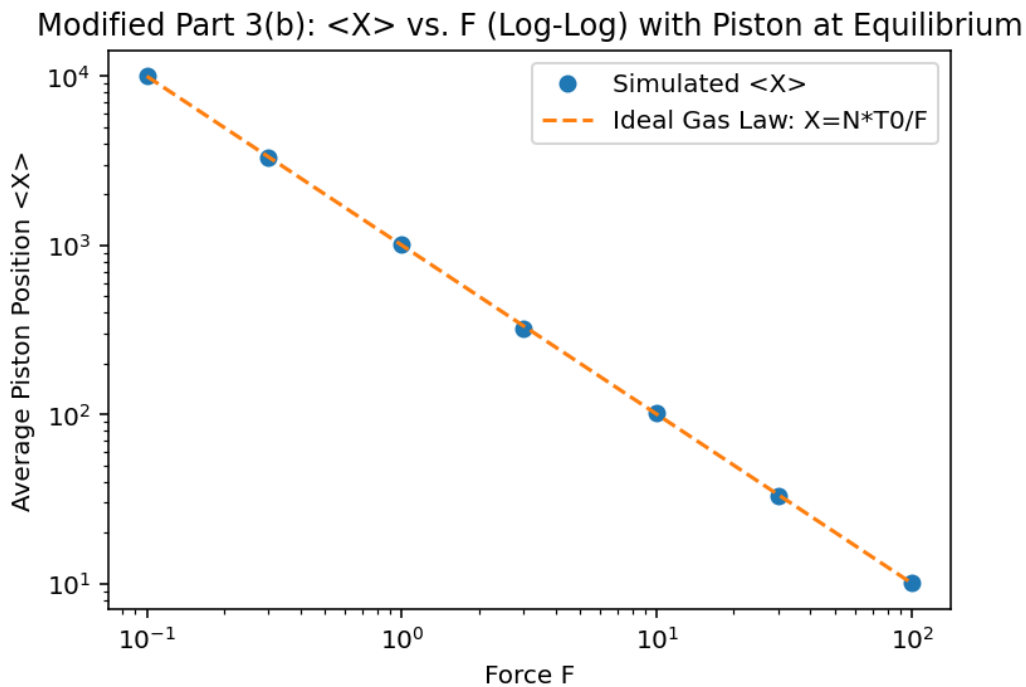


Figure 16: 3b.png

Clearly the graph's above illustrate that the simulation coincides with the ideal gas law.

```

def plot_part_b():
    """
    Modified Part 3(b): For forces F spanning 0.1 to 100, simulate the system and
    compute the
    time-averaged piston position. In this modified version, the simulation
    initializes the piston
    at the equilibrium position (  $X = N \cdot T_0 / F$ ) to help the system settle at the ideal
    gas law prediction.
    """
    forces = np.array([0.1, 0.3, 1, 3, 10, 30, 100])
    avg_positions = []
    theoretical = []
    N = 1000
    m = 1
    M = 100
    T0 = 1
    t_max = 200      # You may need to increase t_max if oscillations are still
                     # significant.
    dt_output = 0.05

    for F in forces:
        # Note: simulate_piston_event_driven now accepts init_piston_at_eq.
        result = simulate_piston_event_driven(N, m, M, T0, F, t_max,
                                              init_vel_dist='maxwell', dt_output=
                                              dt_output,
                                              init_piston_at_eq=True)

        t = result['t']
        X = result['X']
        # Use the final 1/3 of the simulation for averaging
        mask = t > (t.max() * 2/3)
        avg = compute_weighted_average(X[mask], t[mask])
        avg_positions.append(avg)
        theoretical.append(N * T0 / F)

    forces = np.array(forces)
    avg_positions = np.array(avg_positions)
    theoretical = np.array(theoretical)

    plt.figure()
    plt.loglog(forces, avg_positions, 'o', label="Simulated <X>")
    plt.loglog(forces, theoretical, '--', label="Ideal Gas Law:  $X=N \cdot T_0 / F$ ")
    plt.xlabel("Force F")
    plt.ylabel("Average Piston Position <X>")
    plt.title("Modified Part 3(b): <X> vs. F (Log-Log) with Piston at Equilibrium")
    plt.legend()
    plt.show()

```

(c)

Here we can plot the distributions of particle velocities for different number of collisions counts.

Initially (No collisions), we have a pure bimodal distribution while as we increase the collisions we clearly see the formation of a Gaussian distribution. This is made evident by the plot below. The particles with higher velocities are more likely to hit off the piston and thus reduce their velocities, similarly smaller velocities are less likely to have collisions. This leads to the Maxwell distribution.

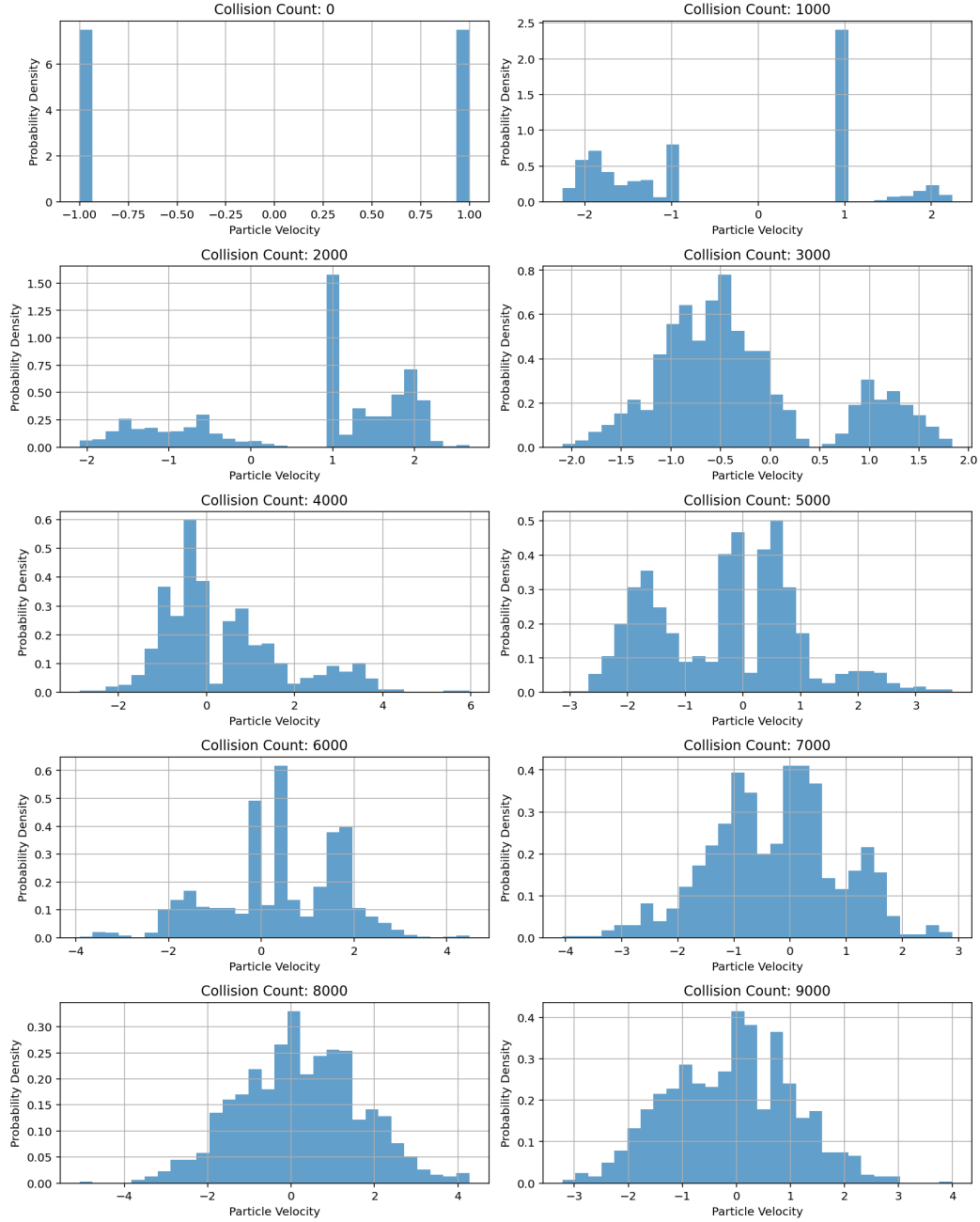


Figure 17: Here we have the plot of the Probability density versus the particle velocities for given numbers of collisions.

We can see the resemblance here. This plot tells us that we are much more likely to find a particle with a small magnitude of velocity than we are to find a particle with a large one. We are limited here as we can only see a snapshot for a given number of collisions. However, we can visualise this in a 3D plot to encapsulate what is going on.

Despite no interactions thermalization is observed as when each particle collides with the piston this redistributes the energy amongst the particles. As we get many collisions we see the formation of the Maxwell-Boltzmann distribution. So it is the effect of the piston that causes us to observe this thermalization, namely the transformation from a bimodal distribution into a Maxwell Boltzmann distribution.

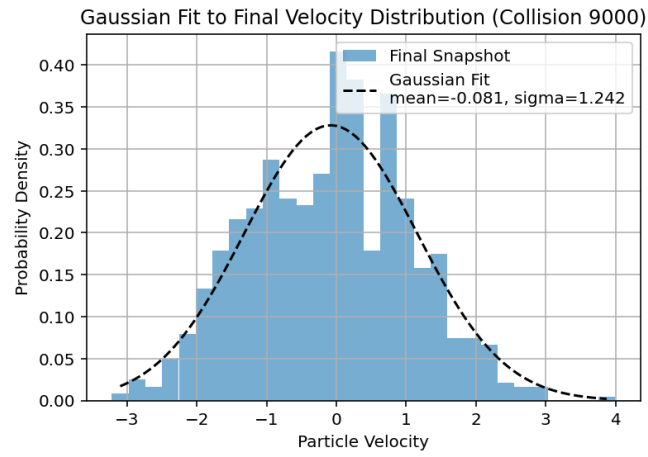


Figure 18: Gaussian fit over the particle velocity distribution for the large collision case (9000) collisions.

3D Plot: Velocity Distribution vs. Collisions

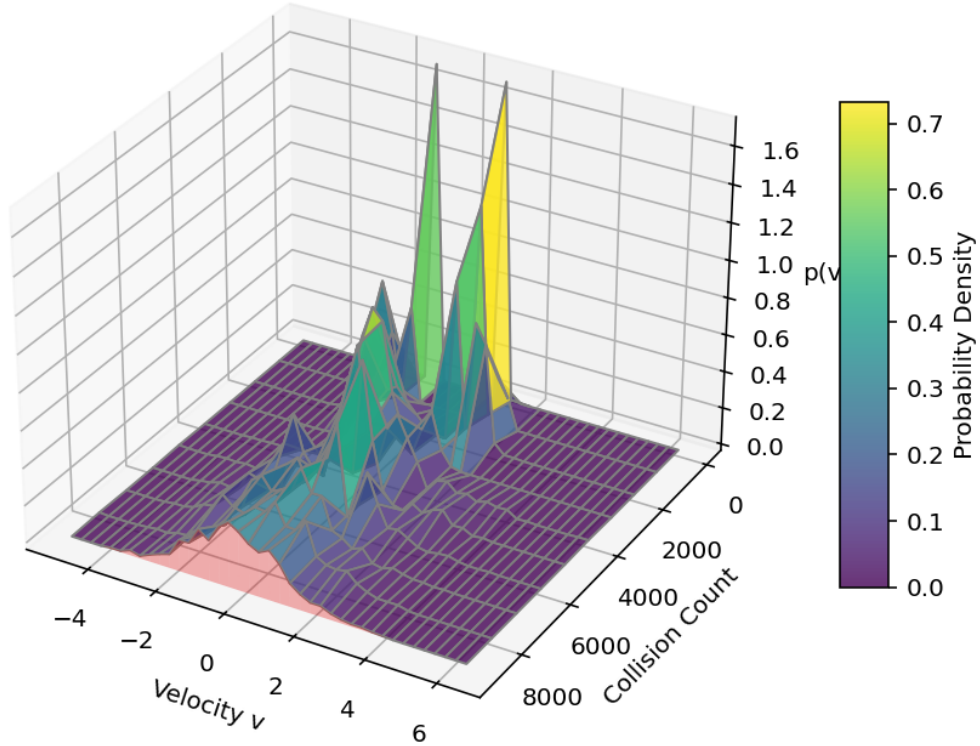


Figure 19: 3D Plot of particle velocity versus the number of collisions versus the Probability of finding a particle.

```

def plot_part_c():
    """
    Part 3(c): Initialize with a bipolar velocity distribution and record velocity
    histograms
    at 7 time snapshots. Fit a Gaussian to the final histogram.
    """
    N = 1000; m = 1; M = 100; T0 = 1; F = 10; t_max = 200; dt_output = 0.05
    result = simulate_piston_event_driven(N, m, M, T0, F, t_max,
                                         init_vel_dist='bipolar', dt_output=dt_output
                                         )

    snapshots = result['snapshots']
    times = sorted(snapshots.keys())

    plt.figure(figsize=(12, 8))
    for t_snap in times:
        v = snapshots[t_snap]
        plt.hist(v, bins=30, density=True, alpha=0.5, label=f"t={t_snap:.1f}")
    plt.xlabel("Particle Velocity v")
    plt.ylabel("Probability Density")
    plt.title("Part 3(c): Evolution of Velocity Distribution")
    plt.legend()
    plt.grid(True)

    # Fit a Gaussian to the final snapshot
    v_final = snapshots[times[-1]]
    hist, bin_edges = np.histogram(v_final, bins=30, density=True)
    bin_centers = 0.5*(bin_edges[:-1] + bin_edges[1:])
    try:
        popt, _ = curve_fit(gaussian, bin_centers, hist, p0=[1, 0, np.std(v_final)])
        x_fit = np.linspace(bin_centers.min(), bin_centers.max(), 100)
        plt.plot(x_fit, gaussian(x_fit, *popt), 'k--', label="Gaussian Fit (final)")
        plt.legend()
        print(f"Final Gaussian fit parameters: mean = {popt[1]:.3f}, sigma = {popt
            [2]:.3f}")
    except Exception as e:
        print("Gaussian fit failed:", e)
    plt.show()

```

(d)

(i)

Here we have graphs of enthalpy and the deviation of the enthalpy from the mean. We can see that the enthalpy of the system is approximately constant. We have a deviation of the order 10^{-11} .

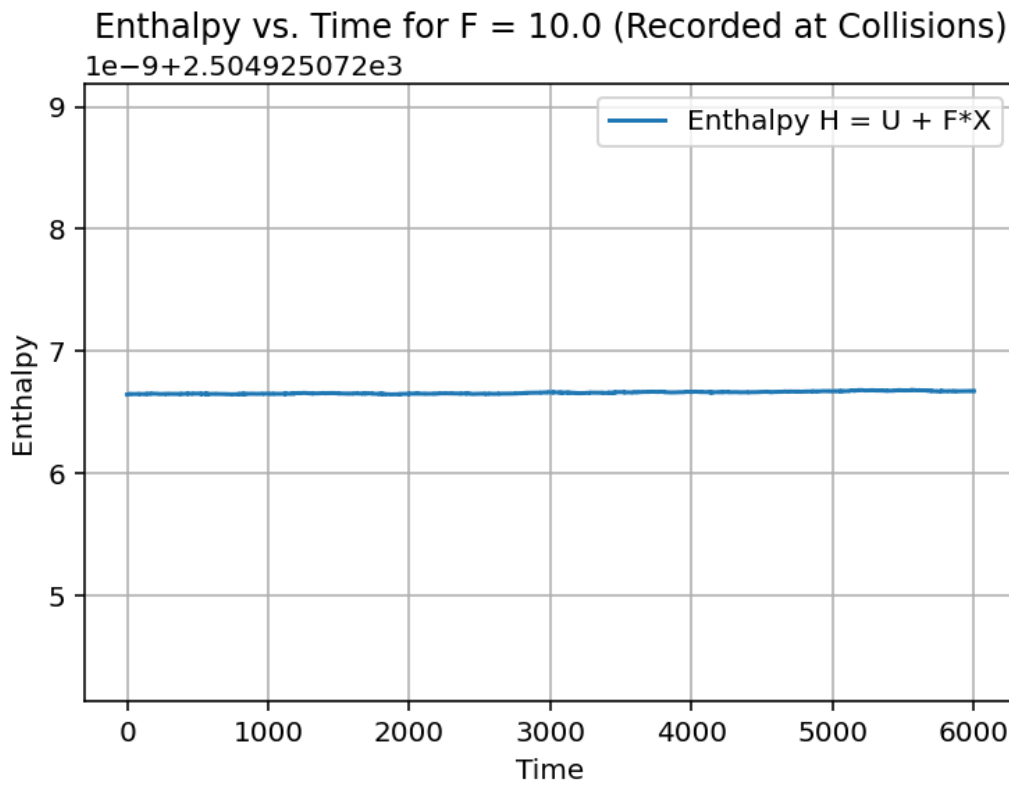


Figure 20: Plot of Enthalpy vs Time. Here we have a horizontal straight line indicating that the enthalpy is conserved.

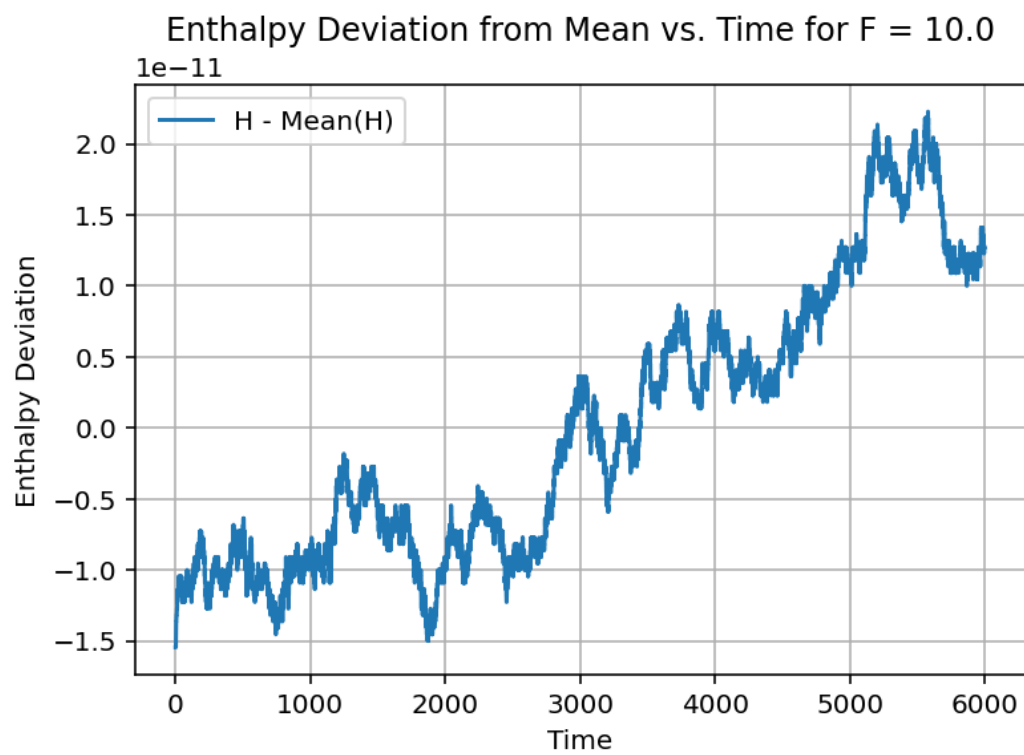


Figure 21: Plot of the Enthalpy deviation from the Mean

```

def plot_part_d():
    """
    Part 3(d): dimensionless enthalpy vs time + priority queue demonstration
    """
    N = 1000
    m = 1
    M = 100
    T0 = 1
    F = 10
    t_max = 6000

    # Record at collision events
    result = simulate_piston_event_driven(
        N, m, M, T0, F, t_max,
        init_vel_dist='maxwell',
        dt_output=0.05,
        record_at_collision_events=True
    )
    t = result['t']
    H = result['enthalpy']

    plt.figure()
    plt.plot(t, H, label="H = U + F*X")
    plt.xlabel("Time (dimensionless)")
    plt.ylabel("Enthalpy (dimensionless)")
    plt.title("Enthalpy vs Time (F=10.0, dimensionless)")
    plt.legend()
    plt.grid(True)
    plt.show()

    H_mean = np.mean(H)
    H_dev = H - H_mean

    plt.figure()
    plt.plot(t, H_dev, label="H - Mean(H)")
    plt.xlabel("Time (dimensionless)")
    plt.ylabel("Deviation of Enthalpy")
    plt.title("Enthalpy Deviation from Mean vs Time")
    plt.legend()
    plt.grid(True)
    plt.show()

    # Priority queue version
    t_end, event_count = simulate_piston_priority_queue(
        1000, m, M, T0, F, t_max
    )
    print("Priority queue simulation done; events processed =", event_count)

```

(ii)

According to 'Atkins, P. and de Paula, J. (2006). Physical Chemistry (8th ed.). Oxford University Press.' At standard temperature and pressure, the mean free path of air molecules is on the order of tens of nanometers. To have "non-interacting" molecules in a 1D container, we'd need the container size to be comparable to or smaller than the mean free path(10^{-8} to $10^{-7}m$), or a much lower pressure so that molecules rarely collide with each other. In reality, for a macroscopic container at 1 atm, molecules do collide frequently, so the truly non-interacting assumption wouldn't strictly hold.

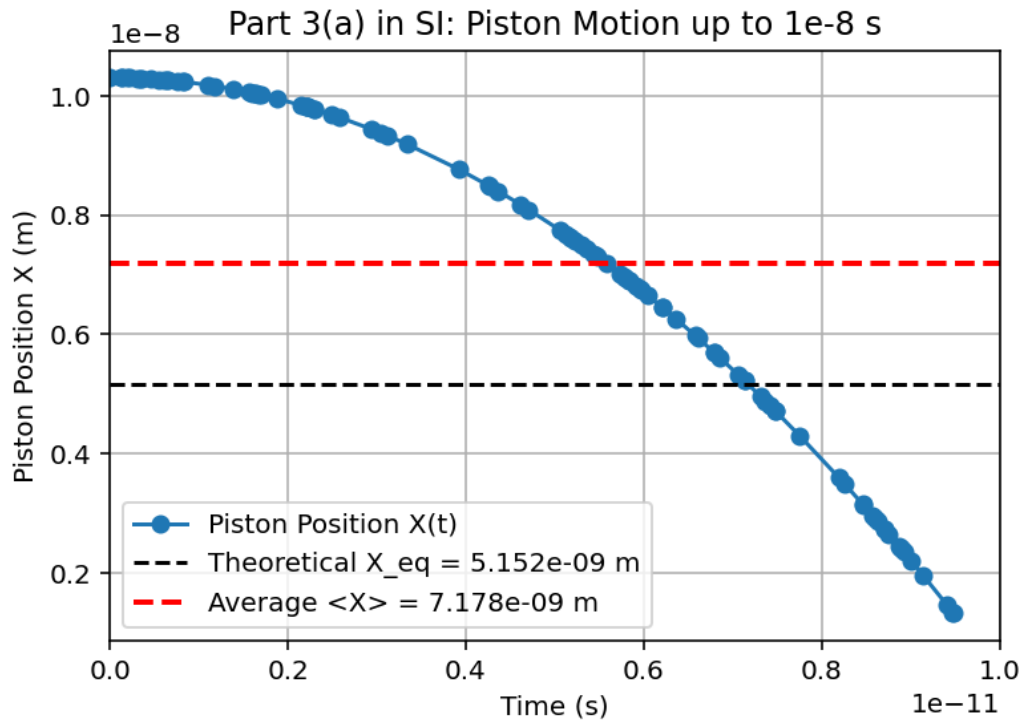


Figure 22: Incorrect graph for the SI units section. I'm not sure why but this is the graph I got despite only changing the parameters so they have dimensions. There is an issue with the time as well as we can see the order of magnitude doesn't match the expected values.

```
def plot_part_a_SI():
    """
    Part 3(a) in SI units:
    We want to see up to 1e-8 s on the time axis.
    """
    # Physical & numerical parameters
    k_B = 1.380649e-23
    N = 1000
    m = 5e-26
    M = 100*m
    T0 = 373.15
    F = 10e-10

    # Set t_max to 1e-8 so the simulation doesn't exceed that
    t_max = 1e-9

    # Use a smaller dt_output than t_max to get multiple data points
    dt_output = 1e-10

    result = simulate_piston_event_driven(
        N, m, M, T0, F, t_max,
        init_vel_dist='maxwell',
        dt_output=dt_output,
        k_B=k_B
    )
    t = result['t']
    X = result['X']
    X_eq = result['X_eq']
    avg_X = np.mean(X)
```

```

print(f"Final time: {t[-1]} s (should be <= 1e-8)")
print(f"Avg X = {avg_X} m, Theoretical X_eq = {X_eq} m")

plt.figure()
plt.plot(t, X, 'o-', label="Piston Position X(t)")
plt.axhline(X_eq, color='k', linestyle='--',
            label=f"Theoretical X_eq = {X_eq:.3e} m")
plt.xlabel("Time (s)")
plt.ylabel("Piston Position X (m)")
plt.title("Part 3(a) in SI: Piston Motion up to 1e-8 s")
plt.axhline(avg_X, color='red', linestyle='--', linewidth=2,
            label=f"Average <X> = {avg_X:.3e} m")

# Force the x-axis from 0 to 1e-8
plt.xlim(0, 1e-11)

plt.legend()
plt.grid(True)
plt.show()

```

(iii)

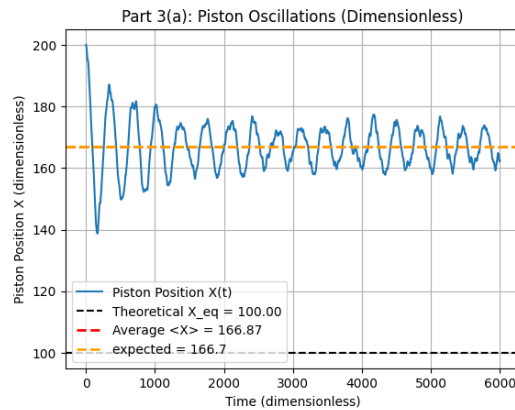


Figure 23: 3a1.png

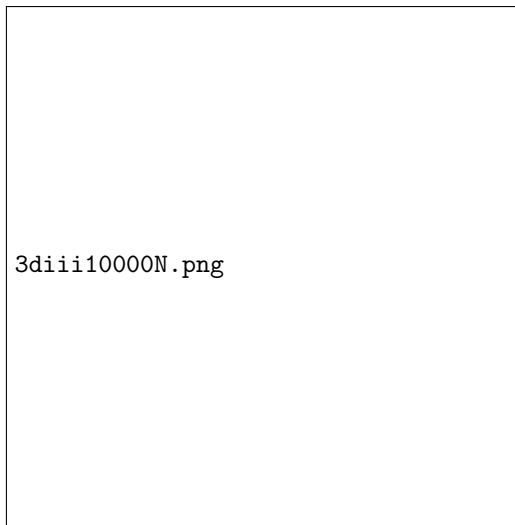


Figure 24: Plot of oscillations for $N = 10000$ Here we observe a potentially asymptotical relationship but it is difficult to say for sure.

As you can see from the graphs above as we increase the time that the simulation is ran for the better it models an ideal gas. One could imagine as they improve their hardware and increase the time the closer this gets to being a perfect model.

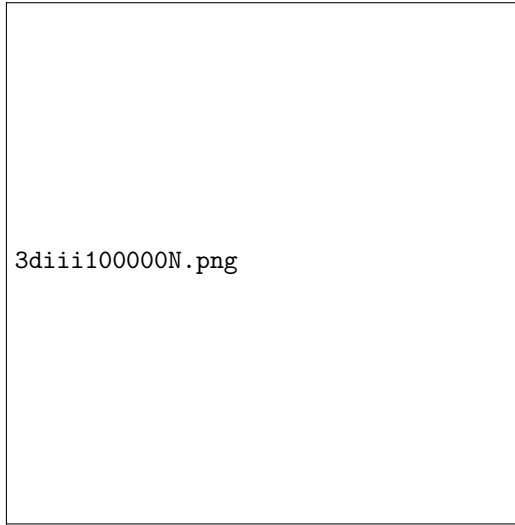


Figure 25: Plot of oscillations for $N = 100000$ Here we observe a clear an asymptotical relationship.

(iv)

The Lenard Jones potential was implemented to introduce a weak interaction between the particles. The strenght of this interaction depends on the parameter ϵ .

In 3D this potential is given by

$$F(r) = 24\epsilon \left(2 \left(\frac{\sigma}{r} \right)^{12} - \left(\frac{\sigma}{r} \right)^6 \right) \frac{1}{r} \quad (61)$$

where for a pair of two particles (i, j) $r_{ij} = x_j - x_i$

```
lennard_jones_force_1d(x, epsilon, sigma):
    """
    1D Lennard-Jones force on each particle (O(N^2)).
    F_ij = 24 * epsilon * [ 2 * (sigma^12/r^13) - (sigma^6/r^7) ]
    """
    N = len(x)
    F = np.zeros(N, dtype=float)
    for i in range(N):
        for j in range(i+1, N):
            r_ij = x[j] - x[i]
            if abs(r_ij) < 1e-12:
                continue
            r2 = r_ij**2
            sr2 = (sigma**2) / r2
            sr6 = sr2**3
            sr12 = sr6**2
            f_mag = 24.0 * epsilon * (2*sr12 - sr6) / r2
            fx = f_mag if r_ij > 0 else -f_mag
            F[i] -= fx
            F[j] += fx
    return F
```

Figure 26: Here the implementation of a LJ force is displayed. The positions are looped and r is determined for $r < 10^{-12}$. Then the magnitude of the force is found using the formula and the direction is determined depending on the sign of r . We see that the interaction scales with the value of ϵ

(v)

The efficiency is improved by storing each particles collision time and only updating affected particles after a collision. This saves on the computation time as we are reducing the number of calculations computed for the system.

```
def simulate_piston_priority_queue_fast(N, m, M, T0, F, t_max, k_B=1.0):
    """
    Efficient event-driven simulation using a priority queue (fast version) to reduce
    the number
    of particles evaluated per event.
```

Each particle's next collision time is computed once and stored in a min-heap. After processing a collision, only the affected particle's event is updated.

Returns:

```

    t: final simulation time
    event_count: total number of events processed
"""
X_eq = N * k_B * T0 / F
X0 = 2 * X_eq
X = X0
piston_v = 0.0
a_piston = -F / M

# Initialize particle positions and velocities
positions = np.random.uniform(0, X, size=N)
sigma = np.sqrt(k_B * T0 / m)
velocities = np.random.normal(0.0, sigma, size=N)
t = 0.0
event_count = 0

# Priority queue: Each event is a tuple (collision_time, event_type,
    particle_index)
# event_type: 0 => collision with wall; 1 => collision with piston.
events = []
for i in range(N):
    dt_wall = -positions[i] / velocities[i] if velocities[i] < 0 else np.inf
    dt_pist = quadratic_solver(0.5 * a_piston, piston_v - velocities[i], X -
        positions[i]) if velocities[i] > 0 else np.inf
    dt = min(dt_wall, dt_pist)
    if dt < np.inf:
        event_type = 0 if dt_wall <= dt_pist else 1
        events.append((t + dt, event_type, i))
heapq.heapify(events)

start_time = time.time()
while t < t_max and events:
    event_time, event_type, i = heapq.heappop(events)
    dt = event_time - t

    # Advance system state
    positions += velocities * dt
    X += piston_v * dt + 0.5 * a_piston * (dt**2)
    piston_v += a_piston * dt
    t = event_time

    # Process collision event for particle i
    if event_type == 0:
        # Collision with left wall
        velocities[i] = -velocities[i]
    else:
        # Collision with piston: perform elastic collision
        u = velocities[i]
        U = piston_v
        new_u, new_U = elastic_collision(u, U, m, M)
        velocities[i] = new_u
        piston_v = new_U
        positions[i] = min(positions[i], X)
    event_count += 1

    # Recompute next collision time for particle i only
    if velocities[i] < 0:

```

```

        dt_wall = -positions[i] / velocities[i]
    else:
        dt_wall = np.inf
    if velocities[i] > 0:
        dt_pist = quadratic_solver(0.5 * a_piston, piston_v - velocities[i], X -
            positions[i])
    else:
        dt_pist = np.inf
    dt_new = min(dt_wall, dt_pist)
    if dt_new < np.inf:
        new_event_type = 0 if dt_wall <= dt_pist else 1
        heapq.heappush(events, (t + dt_new, new_event_type, i))

elapsed = time.time() - start_time
print(f"Fast Priority queue sim finished. Processed {event_count} events in {
    elapsed:.3f} seconds.")
return t, event_count

```

(e)

ChatGPT was used for some constituents of this programming assignment.

Initially, after reading the instructions I was unsure as to implementing an efficient manor for the assignment as I had a general idea of defining functions and calling them for the different parts. GPT suggested making some initial Utility functions which I created.

GPT was used as a quicker way of accessing documentation for libraries such as numpy. Rather than going onto the documentation of libraries when wanting to do something I asked GPT for the link to relevant docs.

To check for physical reasonableness of the results I ensured that the oscillations dampened in time towards X_{eq} , while the velocity distributions matched the Maxwell distribution. Also the simulation agreed with the theoretical value of 166.67.

- I calculated the time-weighted average piston position over several oscillations and plotted these averages for various force values on log-log axes. The trend was checked against the ideal gas law, confirming that the simulation agrees with the theoretical prediction.
- I generated velocity histograms starting from the bipolar distribution and verified that the particle velocities evolve into a Maxwell-Boltzmann distribution. I fitted Gaussian curves to the histograms and verified that the mean and standard deviation were consistent with the equipartition theorem.
- I checked energy conservation by plotting the enthalpy $H = U + FX$ versus time, ensuring that it remains roughly constant as the system approaches equilibrium.